

ABSTRACTION

```
using System;
```

```
using System.Collections.Generic;
```

```
interface IDepartment
```

```
{
```

```
    void PrintDepartmentDetails();
```

```
}
```

```
class Hostel
```

```
{
```

```
    public string HostelName { get; private set; }
```

```
    public string HostelLocation { get; private set; }
```

```
    public int NumberOfRooms { get; private set; }
```

```
    public void SetHostelDetails(string name, string location, int rooms)
```

```
{
```

```
        HostelName = name;
```

```
        HostelLocation = location;
```

```
        NumberOfRooms = rooms;
```

```
}
```

```
    public void PrintHostelDetails()
```

```
{
```

```
        Console.WriteLine($"Hostel Name: {HostelName}");
```

```
        Console.WriteLine($"Hostel Location: {HostelLocation}");
```

```
        Console.WriteLine($"Number of Rooms: {NumberOfRooms}");
```

```
}
```

```
}
```

```
class Student : Hostel, IDepartment
```

```
{
```

```
    public string StudentName { get; private set; }
```

```
    public string RegdNo { get; private set; }
```

```
    public string ElectiveSubject { get; private set; }
```

```
    public double AvgMarks { get; private set; }
```

```
    public string DeptName { get; private set; }
```

```
    public string DeptHead { get; private set; }
```

```
    public void GetStudentDetails()
```

```
    {
```

```
        Console.Write("Enter Student Name: ");
```

```
        StudentName = Console.ReadLine();
```

```
        Console.Write("Enter Registration Number: ");
```

```
        RegdNo = Console.ReadLine();
```

```
        Console.Write("Enter Elective Subject: ");
```

```
        ElectiveSubject = Console.ReadLine();
```

```
        Console.Write("Enter Average Marks: ");
```

```
        AvgMarks = Convert.ToDouble(Console.ReadLine());
```

```
        Console.Write("Enter Department Name: ");
```

```
        DeptName = Console.ReadLine();
```

```
        Console.Write("Enter Department Head: ");
```

```
        DeptHead = Console.ReadLine();
```

```
Console.Write("Enter Hostel Name: ");  
string hostelName = Console.ReadLine();
```

```
Console.Write("Enter Hostel Location: ");  
string hostelLocation = Console.ReadLine();
```

```
Console.Write("Enter Number of Rooms in Hostel: ");  
int numberOfRooms = Convert.ToInt32(Console.ReadLine());  
SetHostelDetails(hostelName, hostelLocation, numberOfRooms);  
}
```

```
public void PrintStudentDetails()  
{  
    Console.WriteLine("--- Student Details ---");  
    Console.WriteLine($"Student Name: {StudentName}");  
    Console.WriteLine($"Registration Number: {RegdNo}");  
    Console.WriteLine($"Elective Subject: {ElectiveSubject}");  
    Console.WriteLine($"Average Marks: {AvgMarks}");  
    PrintDepartmentDetails();  
    PrintHostelDetails();  
}
```

```
public void PrintDepartmentDetails()  
{  
    Console.WriteLine($"Department Name: {DeptName}");  
    Console.WriteLine($"Department Head: {DeptHead}");  
}  
}
```

```
class UniversityDriverMenu
{
    private List<Student> studentList = new List<Student>();

    public void Start()
    {
        int choice;
        do
        {
            Console.WriteLine("\n--- University Management System ---");
            Console.WriteLine("1. Admit New Student");
            Console.WriteLine("2. Migrate a Student");
            Console.WriteLine("3. Display Student Details");
            Console.WriteLine("4. Exit");
            Console.Write("Enter your choice: ");
            choice = Convert.ToInt32(Console.ReadLine());

            switch (choice)
            {
                case 1:
                    AdmitNewStudent();
                    break;
                case 2:
                    MigrateStudent();
                    break;
                case 3:
                    DisplayStudentDetails();
                    break;
                case 4:
                    Console.WriteLine("Exiting... Goodbye!");
                    break;
            }
        } while (choice != 4);
    }
}
```

```

        default:
            Console.WriteLine("Invalid choice. Please try again.");
            break;
    }
} while (choice != 4);
}

private void AdmitNewStudent()
{
    Student student = new Student();
    student.GetStudentDetails();
    studentList.Add(student);
    Console.WriteLine("Student admitted successfully!");
}

private void MigrateStudent()
{
    Console.Write("Enter Registration Number of the student to migrate: ");
    string regdNo = Console.ReadLine();

    foreach (var student in studentList)
    {
        if (student.RegdNo == regdNo)
        {
            Console.Write("Enter New Hostel Name: ");
            string newHostelName = Console.ReadLine();

            Console.Write("Enter New Hostel Location: ");
            string newHostelLocation = Console.ReadLine();

            Console.Write("Enter Number of Rooms in New Hostel: ");

```

```

        int newNumberOfRooms = Convert.ToInt32(Console.ReadLine());

        student.SetHostelDetails(newHostelName, newHostelLocation, newNumberOfRooms);
        Console.WriteLine("Student hostel details updated successfully!");
        return;
    }
}

Console.WriteLine($"Student with Registration Number {regdNo} not found.");
}

private void DisplayStudentDetails()
{
    Console.Write("Enter Registration Number of the student: ");
    string regdNo = Console.ReadLine();

    foreach (var student in studentList)
    {
        if (student.RegdNo == regdNo)
        {
            student.PrintStudentDetails();
            return;
        }
    }

    Console.WriteLine($"Student with Registration Number {regdNo} not found.");
}
}

```

```

class Program
{
    static void Main(string[] args)

```

```
{  
    UniversityDriverMenu driverMenu = new UniversityDriverMenu();  
    driverMenu.Start();  
}  
}
```

ENCAPSULATION

```
using System;
```

```
class CommissionEmployee
```

```
{
```

```
    public string FirstName { get; private set; }
```

```
    public string LastName { get; private set; }
```

```
    public string SocialSecurityNumber { get; private set; }
```

```
    private double grossSales;
```

```
    private double commissionRate;
```

```
    public CommissionEmployee(string firstName, string lastName, string socialSecurityNumber,  
double grossSales, double commissionRate)
```

```
    {
```

```
        FirstName = firstName;
```

```
        LastName = lastName;
```

```
        SocialSecurityNumber = socialSecurityNumber;
```

```
        SetGrossSales(grossSales);
```

```
        SetCommissionRate(commissionRate);
```

```
    }
```

```
    public double GetGrossSales() => grossSales;
```

```
    public void SetGrossSales(double grossSales)
```

```
    {
```

```
        if (grossSales < 0.0)
```

```
            throw new ArgumentException("Gross sales must be >= 0.0");
```

```
        this.grossSales = grossSales;
```

```
    }
```



```

public double GetCommissionRate() => commissionRate;

public void SetCommissionRate(double commissionRate)
{
    if (commissionRate < 0.0 || commissionRate > 1.0)
        throw new ArgumentException("Commission rate must be between 0.0 and 1.0");
    this.commissionRate = commissionRate;
}

public double Earnings() => grossSales * commissionRate;

public override string ToString()
{
    return $"Commission Employee: {FirstName} {LastName}\n" +
        $"Social Security Number: {SocialSecurityNumber}\n" +
        $"Gross Sales: {grossSales:F2}\n" +
        $"Commission Rate: {commissionRate:F2}";
}
}

class Program
{
    static void Main()
    {
        try
        {
            // Create a CommissionEmployee object
            CommissionEmployee employee = new CommissionEmployee("Edem", "Kojo", "88347-5639",
60000.0, 0.5);

            Console.WriteLine("Initial Employee Details:\n" + employee);

```

```

        // Update grossSales and commissionRate
        employee.SetGrossSales(15000.0);
        employee.SetCommissionRate(0.2);

        Console.WriteLine("\nUpdated Employee Details:\n" + employee);
        Console.WriteLine($"Employee's Earnings: {employee.Earnings():F2}");

        // Test validation for grossSales
        Console.WriteLine("\nTesting invalid grossSales...");
        employee.SetGrossSales(-5000.0);
    }
    catch (ArgumentException e)
    {
        Console.WriteLine("Exception caught: " + e.Message);
    }

    try
    {
        // Test validation for commissionRate
        Console.WriteLine("\nTesting invalid commissionRate...");

        CommissionEmployee invalidEmployee = new CommissionEmployee("Jane", "Smith", "987-65-
4321", 5000.0, 1.5);
    }
    catch (ArgumentException e)
    {
        Console.WriteLine("Exception caught: " + e.Message);
    }
}

```

INHERITANCE

```
using System;
```

```
using System.Collections.Generic;
```

```
abstract class Vehicle
```

```
{
```

```
    public string VehicleId { get; private set; }
```

```
    public string Brand { get; private set; }
```

```
    public string Model { get; private set; }
```

```
    public bool IsAvailable { get; private set; }
```

```
    public Vehicle(string vehicleId, string brand, string model)
```

```
    {
```

```
        VehicleId = vehicleId;
```

```
        Brand = brand;
```

```
        Model = model;
```

```
        IsAvailable = true;
```

```
    }
```

```
    public void RentVehicle()
```

```
    {
```

```
        if (!IsAvailable)
```

```
            throw new Exception("Vehicle is already rented.");
```

```
        IsAvailable = false;
```

```
    }
```

```
    public void ReturnVehicle()
```

```
    {
```

```
        IsAvailable = true;
```

```
}
```

```
public abstract double CalculateRentalCost(int days);
```

```
}
```

```
class Bike : Vehicle
```

```
{
```

```
    public int EngineCapacity { get; private set; }
```

```
    public Bike(string vehicleId, string brand, string model, int engineCapacity)
```

```
        : base(vehicleId, brand, model)
```

```
{
```

```
    EngineCapacity = engineCapacity;
```

```
}
```

```
    public override double CalculateRentalCost(int days)
```

```
{
```

```
        return 500 * days + EngineCapacity / 10.0;
```

```
}
```

```
}
```

```
class Car : Vehicle
```

```
{
```

```
    public int SeatingCapacity { get; private set; }
```

```
    public Car(string vehicleId, string brand, string model, int seatingCapacity)
```

```
        : base(vehicleId, brand, model)
```

```
{
```

```
    SeatingCapacity = seatingCapacity;
```

```

    }

    public override double CalculateRentalCost(int days)
    {
        return 1000 * days + SeatingCapacity * 50;
    }
}

class RentalManager
{
    private List<Vehicle> vehicles;

    public RentalManager()
    {
        vehicles = new List<Vehicle>();
    }

    public void AddVehicle(Vehicle vehicle)
    {
        vehicles.Add(vehicle);
    }

    public void DisplayAvailableVehicles()
    {
        Console.WriteLine("Available Vehicles:");
        foreach (var vehicle in vehicles)
        {
            if (vehicle.IsAvailable)
                Console.WriteLine($"{vehicle.VehicleId} - {vehicle.Brand} {vehicle.Model}");
        }
    }
}

```

```
}  
}
```

class Program

```
{  
    static void Main()  
    {  
        RentalManager rentalManager = new RentalManager();  
  
        Car car1 = new Car("C1", "Tesla", "Cybertruck", 5);  
        Car car2 = new Car("C2", "Toyota", "Corolla", 4);  
        Bike bike1 = new Bike("B1", "Haojue", "Aloba 125", 150);  
        Bike bike2 = new Bike("B2", "Yamaha", "MT-07", 200);  
  
        rentalManager.AddVehicle(car1);  
        rentalManager.AddVehicle(car2);  
        rentalManager.AddVehicle(bike1);  
        rentalManager.AddVehicle(bike2);  
  
        rentalManager.DisplayAvailableVehicles();  
  
        Console.WriteLine($"Rental cost for C1 for 3 days: {car1.CalculateRentalCost(3)}");  
        rentalManager.DisplayAvailableVehicles();  
    }  
}
```

POLYMORPHISM

```
using System;
```

```
using System.Collections.Generic;
```

```
abstract class Vehicle
```

```
{
```

```
    public string VehicleId { get; private set; }
```

```
    public string Model { get; private set; }
```

```
    public double FuelLevel { get; private set; }
```

```
    public Vehicle(string vehicleId, string model, double fuelLevel)
```

```
    {
```

```
        VehicleId = vehicleId;
```

```
        Model = model;
```

```
        FuelLevel = fuelLevel;
```

```
    }
```

```
    public void Refuel(double liters)
```

```
    {
```

```
        FuelLevel += liters;
```

```
        Console.WriteLine($"Refueled {liters} liters. New fuel level: {FuelLevel} liters.");
```

```
    }
```

```
    public abstract void CalculateRange();
```

```
}
```

```
class Car : Vehicle
```

```
{
```

```

private double fuelEfficiency;

public Car(string vehicleId, string model, double fuelLevel, double fuelEfficiency)
    : base(vehicleId, model, fuelLevel)
{
    this.fuelEfficiency = fuelEfficiency;
}

public override void CalculateRange()
{
    double range = FuelLevel * fuelEfficiency;
    Console.WriteLine($"Car {Model} can travel {range} kilometers.");
}
}

class Truck : Vehicle
{
    private double cargoWeight;
    private const double BASE_FUEL_EFFICIENCY = 5.0;

    public Truck(string vehicleId, string model, double fuelLevel, double cargoWeight)
        : base(vehicleId, model, fuelLevel)
    {
        this.cargoWeight = cargoWeight;
    }

    public override void CalculateRange()
    {
        double effectiveFuelEfficiency = BASE_FUEL_EFFICIENCY - (cargoWeight / 1000.0);
        if (effectiveFuelEfficiency <= 0)
    }
}

```



```

    {
        Console.WriteLine($"Truck {Model} cannot operate with cargo over the limit.");
    }
    else
    {
        double range = FuelLevel * effectiveFuelEfficiency;
        Console.WriteLine($"Truck {Model} with {cargoWeight} kg cargo can travel {range}
kilometers.");
    }
}
}
}

```

```

class TransportationManager
{
    public void OperateVehicle(Vehicle vehicle)
    {
        vehicle.CalculateRange();
    }
}

```

```

// Main Program
class Program
{
    static void Main()
    {
        List<Vehicle> vehicles = new List<Vehicle>
        {
            new Car("C001", "Sedan", 50.0, 15.0),
            new Truck("T001", "Freightliner", 100.0, 2000.0),
            new Truck("T002", "Heavy Hauler", 80.0, 10000.0)
        }
    }
}

```

```
};
```

```
TransportationManager manager = new TransportationManager();
```

```
Console.WriteLine("--- Refuel and Calculate Range for Sedan ---");
```

```
vehicles[0].Refuel(10);
```

```
manager.OperateVehicle(vehicles[0]);
```

```
Console.WriteLine("\n--- Calculate Range for Freightliner ---");
```

```
manager.OperateVehicle(vehicles[1]);
```

```
Console.WriteLine("\n--- Calculate Range for Heavy Hauler ---");
```

```
manager.OperateVehicle(vehicles[2]);
```

```
}
```

```
}
```